Markov Decision Process and The Bellman Equation

CSC 480: Artificial Intelligence, Spring 2026
Guest lecture, Samuel Kaplan: Course Notes

Abstract
We will cover Markov Decision Processes and the Bellman equation, then show how value iteration and policy iteration use that structure to compute optimal policies.

Contents

I Markov Decision Process

From the first lecture we have been describing an agent as something that perceives its environment and chooses actions. The agent framework asks us to explore our choices. To evalaute and select actions deliberately, in pursuit of a goal, in a way we can call rational.

In the settings we have studied so far, rationality was easy to define. The agent has a goal, it searches forward through consequences, and the rational choice is whichever action leads there. That works because determinism is doing a lot of the heavy lifting. When every action has a guaranteed outcome, "choose optimally" has a clean answer.

Take away that certainty and the question gets harder. Probability gave us a framework for uncertain worlds, where the agent holds *beliefs* about its environment rather than certain knowledge. But belief alone does not tell us what to do. How does an agent choose *optimally* when it cannot be certain where its actions will lead?

A fixed plan, computed from a single starting point, is no longer enough. Instead of a plan, we want a *policy*: a complete prescription for what to do in every possible situation. Markov Decision Processes give us a framework for computing exactly that.

A totally unrelated example

Suppose it's the night before a given lecture, and our unassuming lecturer was prepping the notes. Our lecturer's goal is to work hard and finish this lecture on time. To help our lecturer out, lets describe this world in three states the lecturer can be in:

- Writing: Actually making progress on the notes.
- Googling: Looking something up "really quickly."
- Doomscrolling: Completely off task, watching videos about whatever the algorithm has decided to serve up tonight.

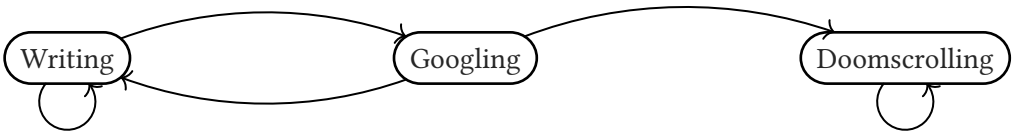

Figure 1: State Transition Diagram for Lecture Preparation

The lecturer has exactly two actions available in both Writing and Googling: *Focus* (the safe, steady option) and *Browse* (the tempting but risky one). From Writing, Focus always keeps you writing. Browse is a coin flip, half the time you find something useful and stay productive, half the time you end up down a Googling rabbit hole. From Googling, Focus gives you even odds of snapping back to Writing or staying stuck. But choosing Browse from Googling? That sends you straight to Doomscrolling.

A Markov Decision Process gives us the formal language to describe exactly this kind of problem. So right now we have the states(Writting, Googling, and Doomscrolling), and the actions(Focus and Browse), so whats next?

> ***Definition: Markov Decision Process***
> A Markov Decision Process (MDP) consists of:
> - A set of *states* S
> - A set of *actions* $A(s)$ available in state $s \in S$
> - A *transition model* $T(s, a, s') = P(s' \mid s, a)$: the probability of landing in state s' after taking action a in state s
> - A *reward function* $R(s, a, s')$ giving the immediate reward for each transition [1]
> - A *discount factor* $\gamma \in [0, 1]$ controlling how much future rewards are valued relative to immediate ones [2]

[1] The reward function is sometimes simplified as $R(s)$ or $R(s, a)$ depending on the problem. The full form $R(s, a, s')$ is the most general.

[2] The definition writes $\gamma \in [0, 1]$ but finite and infinite horizon problems have different requirements. For infinite horizon problems $\gamma = 1$ is not allowed since the sum may diverge, so in practice $\gamma \in [0, 1)$.

Note that for each state-action pair (s, a), the transition probabilities must form a valid probability distribution:

$$\forall s, a : \sum_{s'} T(s, a, s') = 1 \tag{1}$$

For now lets ignore the discount factor and explore our example, the full transition and reward model is:

State	Action	$P'(\text{Writing})$	$P'(\text{Googling})$	$P'(\text{DS})$	Reward
Writing	Focus	1.0	0.0	0.0	$+1$
Writing	Browse	0.5	0.5	0.0	$+2$
Googling	Focus	0.5	0.5	0.0	$+1$
Googling	Browse	0.0	0.0	1.0	$+20$
DS	Browse	0.0	0.0	1.0	-10

Table 1: Transition model $T(s, a, s')$ and rewards for Lecture Preparation

Focus always keeps you on task, from either Writing or Googling it never sends you somewhere worse:

$$T(\text{Writing}, \text{Focus}, \text{Writing}) = 1.0$$
$$T(\text{Googling}, \text{Focus}, \text{Writing}) = 0.5, \quad T(\text{Googling}, \text{Focus}, \text{Googling}) = 0.5$$

Focus yields a modest $+1$ per step. Browse is the risky choice, from Writing it is a coin flip between staying productive and sliding into Googling, and from Googling it sends you straight to Doomscrolling with certainty:

$$T(\text{Writing}, \text{Browse}, \text{Writing}) = 0.5, \quad T(\text{Writing}, \text{Browse}, \text{Googling}) = 0.5$$
$$T(\text{Googling}, \text{Browse}, \text{DS}) = 1.0$$

Browse yields $+2$ from Writing when it goes well, but the catastrophic $R(\text{Googling}, \text{Browse}, \text{DS}) = -10$ makes it a bad idea when Googling.

Updating the diagram with rewards on each state and transition probabilities labeled by action:

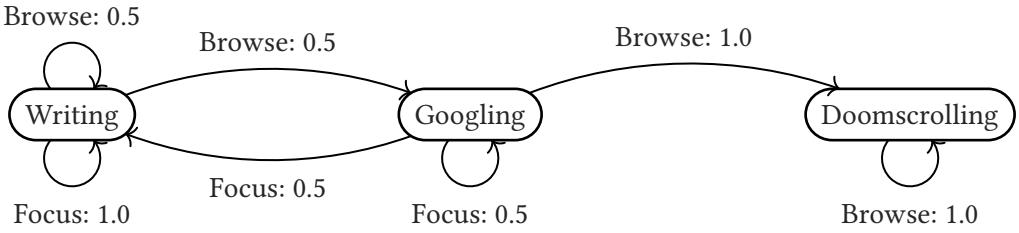

Figure 2: State Transition Diagram with Transition Probabilities

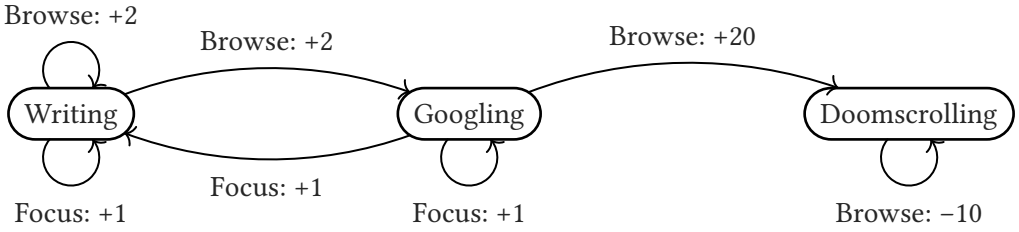

Figure 3: State Transition Diagram with Rewards

The Markov Property

The defining feature of a Markov Decision Process is the *Markov property*, in that the future depends only on the *present state*, not on the history of how the agent arrived there.

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, ..., s_0, a_0) = P(s_{t+1} \mid s_t, a_t) \tag{2}$$

This is a statement of *conditional independence*. Given the current state s_t and action a_t, the next state s_{t+1} is independent of everything that came before. The same logic underpins d-separation in Bayes Networks, where observing a node blocks information flow through it. Here, the current state is a sufficient summary of all relevant history, and that is precisely what makes MDPs tractable, or solvable without the state space blowing up. Rather than reasoning about arbitrarily long decision histories, we only need to reason about where we are right now.

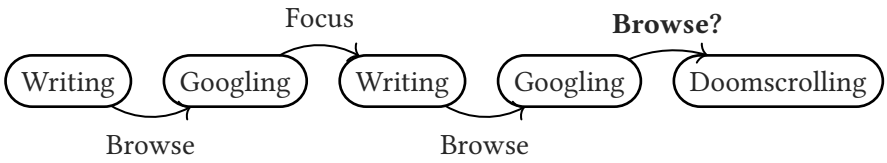

Figure 4: An example iteration

In our example, whether our poor lecturer spirals from Googling into Doomscrolling depends only on being in the Googling state at that exact moment, not on how many times they have previously cycled between Writing and Googling, or how many hours the deadline has been looming.

Policy

In a search problem, the solution is a *sequence of actions*. In an MDP, since the environment is stochastic and the agent may find itself in any state, the solution is instead a *policy*.

> ***Definition: Policy***
> A *policy* $\pi : S \to A$ is a mapping from states to actions. The *optimal policy* π^* maximizes the expected total discounted reward from any starting state. [3]

[3] In modern reinforcement learning, finding π^* is exactly what systems like AlphaGo and RLHF tuned language models are doing, the "policy" is literally the neural network being trained.

The important difference from search is that the solution to an MDP is a *complete plan for every state*, not just from a single start. Wherever the environment drops us(whether we stayed focused, got distracted, or fell headfirst into a Doomscrolling spiral), a policy tells us what to do next.

> ***Discussion***
> What is the optimal policy π^* for this MDP? Is Browse from Googling ever a rational choice given the current model? What would need to change, in the rewards or transition probabilities for it to become so?

A policy assigns actions *per state*, so the optimal action can and often does differ across states. Here: $\pi^*(\text{Writing}) = \text{Browse}$ and $\pi^*(\text{Googling}) = \text{Focus}$. From Writing, Browse earns $+2$ versus $+1$ for Focus, and occasionally landing in Googling is not catastrophic since Focus from Googling recovers to Writing with probability 0.5. Browse from Googling is a completely different story: $T(\text{Googling}, \text{Browse}, \text{DS}) = 1.0$ sends you to a state with no exit and -10 every step forever. No immediate reward, however large, can outweigh that. For Browse from Googling to ever be rational the model would need to

give Doomscrolling a way out (some non-zero $T(\text{DS}, X, \text{Writing})$) or make its reward positive.

Horizons

A key structural choice in any MDP is whether the agent acts for a fixed number of steps or indefinitely.

In a *finite horizon* MDP with horizon T, the agent takes exactly T actions. The optimal policy here can be *non-stationary* in that the right choice at step $t = T - 1$ may differ from the right choice at step $t = 1$, because the agent has less time left to recover from a bad outcome. Our example with a lecture deadline the next day is an example of a finite horizon problem.

In an *infinite horizon* MDP, the agent acts indefinitely with no fixed endpoint. Here the optimal policy is *stationary* in that the same action is optimal in a given state regardless of how many steps have already elapsed, since there is no "time remaining" to track. This is a dramatic simplification. The Bellman equation, which we will get tto shortly, applies to the infinite horizon setting.

Discounting

Infinite horizon MDPs introduce a new problem in that, if the agent collects rewards forever, the sum $\sum_{t=0}^{\infty} R_t$ may diverge. Consider our humble lecturer, if they Focus in Writing indefinitely, collecting $+1$ every step, the undiscounted total is $1 + 1 + 1 + \ldots = +\infty$. If they instead end up Doomscrolling, the total is $(-10) + (-10) + (-10) + \ldots = -\infty$. Both diverge, and comparing $+\infty$ to $-\infty$ to decide which policy is better is not meaningful. We cannot say the writing policy is "$+\infty - (-\infty)$ better" than doomscrolling, since that arithmetic is undefined. The solution is the *discount factor* $\gamma \in [0, 1)$ that we set aside from the MDP definition. Instead of summing raw rewards, the agent maximizes the total *discounted* reward:

$$\sum_{t=0}^{\infty} \gamma^t R_t \tag{3}$$

Since rewards are bounded (say $|R_t| \leq R_{\max}$), this geometric series converges to at most $\frac{R_{\max}}{1-\gamma}$, so values are always finite and well-defined.

There are two useful ways to think about γ. First, as a *preference for sooner rewards*, a reward of $+1$ now is worth more than a reward of $+1$ next step, just as a dollar today is worth more than a dollar next year. Second, as a model of *uncertain termination*. Imagine that after each step, a coin is flipped, with probability $1 - \gamma$ the game simply ends and the agent collects no further reward, and with probability γ it carries on. The reward at step t is then only collected if the agent survives all t preceding flips, which happens with probability γ^t(the factor multiplying that reward in the discounted sum).

The parameter γ directly controls the agent's patience:

- $\gamma = 0$: completely myopic, only the immediate reward matters, future consequences are ignored entirely. Think your average gen α gremlin hedonistically seeking rewards.
- $\gamma \to 1$: increasingly patient, distant future rewards count nearly as much as immediate ones. Think of the lecturer who refuses to Browse from Googling precisely because they know exactly where it leads and they feel the full weight of infinite future procrastination bearing down on each decision.

In our example, the value of being stuck in Doomscrolling forever is:

$$V(\text{DS}) = \sum_{t=0}^{\infty} \gamma^t(-10) = \frac{-10}{1-\gamma} \tag{4}$$

As $\gamma \to 1$ this diverges to $-\infty$, which is precisely why $\gamma < 1$ is required for infinite horizon problems, without it, values are undefined and no meaningful policy comparison is possible.

Discussion

1. When $\lim(\gamma \to 0)$, what is the optimal action from Writing? From Googling? Now consider any $\gamma \in (0, 1)$: is there a value of γ for which Browse from Writing is *not* optimal?
2. Now if $\lim(\gamma \to 1)$, and we use $V^*(s)$ to dennote the total expected discounted reward from state s under the optimal policy, write $V^*(\text{Writing})$ as an equation involving $V^*(\text{Writing})$ and $V^*(\text{Googling})$, then do the same for $V^*(\text{Googling})$. Do you notice anything interesting about these equations?

As γ approaches 0 the agent effectively ignores all future consequences, so from Writing it simply takes whichever action yields the highest immediate reward: Browse ($+2$) beats Focus ($+1$). From Googling, Browse yields $+20$ and Focus yields $+1$, so Browse is the clear choice, especially as the discount factor means that the punishment for Browsing while Doomscrolling is effectively nothing.

Browse from Writing remains optimal for *every* $\gamma \in [0, 1)$, as there is no threshold at which Focus becomes preferable. The reason is that Googling under Focus is not a bad state to be in is that it yields $+1$ per step and returns to Writing with probability 0.5 each step. The extra $+1$ immediate reward from Browse always outweighs the modest cost of occasionally landing in Googling rather than staying in Writing. Browse from Googling, by contrast, is not

rational unless γ is low, since it leads with certainty to Doomscrolling, a state whose value $-10/(1-\gamma)$ is catastrophically negative for any reasonable γ.

Now if we assume a high γ (ie $\lim(\gamma \to 1)$) then the the optimal policy:

$(\pi^*(\text{Writing}) = \text{Browse}, \pi^*(\text{Googling}) = \text{Focus}, \pi^*(\text{DS}) = \text{Browse}$[4]$)$

[4]Since Browse is our only option is must be the optimal option.

Which we can then use to create the following equasions:

$$\begin{aligned} V(\text{Writing}) &= 2 + \gamma(0.5 \cdot V(\text{Writing}) + 0.5 \cdot V(\text{Googling})) \\ V(\text{Googling}) &= 1 + \gamma(0.5 \cdot V(\text{Writing}) + 0.5 \cdot V(\text{Googling})) \end{aligned} \quad (5)$$

These are two simultaneous linear equations each with two unknowns, and they can be solved directly. These equations make each state equal the immediate reward of the optimal action plus the discounted expected value of the states it leads to. This recursive structure, with value defined in terms of other values, is the core concept behind *dynamic programming*[5]. Rather than searching over the exponentially many possible action sequences, dynamic programming breaks the problem into overlapping subproblems and once we know the value of every state, we know the optimal action from every state, and those values can be computed by solving the system of equations above . Generalizing it and using it to compute optimal policies is exactly what we will do next.

[5]Richard Bellmanm, who lends his name to the next section, coined the term "dynamic programming" deliberately to obscure the mathematical research he was doing. https://en.wikipedia.org/wiki/Dynamic_programming#History_of_the_name

II The Bellman Equation

In the previous section we wrote down the value equations for Writing and Googling under a specific policy. But there was something unsatisfying about that derivation, we had to know the optimal policy first (Browse from Writing, Focus from Googling) before we could write the equations. In general we do not know the optimal policy, that is precisely what we are trying to find. The Bellman equation gives us a way to express the optimal values without assuming we already know the answer.

Value and Action-Value

We define $V^*(s)$ as the *optimal value* of state s, the expected total discounted reward an agent collects starting from s and acting optimally from that point forward. This is the quantity we want to compute.

Now, the value of a state depends entirely on what the agent does there. The value of taking a specific action a from state s, and then acting optimally afterward, is called the *action value* or *Q-value*:

> ***Definition: Q-value***
>
> $$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')] \tag{6}$$
>
> The Q-value $Q^*(s,a)$ is the expected discounted reward of taking action a in state s and then following the optimal policy from every subsequent state.

The Q-value is the *expected value* of $(R(s,a,s') + \gamma V^*(s'))$ over all possible next states s'. The transition probabilities $T(s,a,s')$ sum to 1 over all s' as they are the weights of a probability distribution. Since the agent cannot control which s' it actually lands in, the Q-value must weigh each possible outcome by how likely it is.

The optimal value of a state is then simply the Q-value of the best available action:

$$V^*(s) = \max_a Q^*(s,a) \tag{7}$$

And the optimal policy just picks that best action:

$$\pi^*(s) = \arg\max_a Q^*(s,a) \tag{8}$$

The Bellman Equation

Substituting the definition of Q^* into $V^*(s) = \max_a Q^*(s,a)$ gives the *Bellman equation*:

> ***Definition: Bellman Equation***
>
> $$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')] \tag{9}$$

Read this equation carefully. The left-hand side is the value of state s. The right-hand side picks the best action (the $\max_a$), then takes the expectation over where that action leads (the $\sum_{s'} T(s,a,s')$), collecting the immediate reward R and the discounted value of the next state $\gamma V^*(s')$. V^* appears on both sides as the equation is recursive.

Value Iteration

This might look circular, but it is not a contradiction. For a finite state space with $\gamma < 1$, the Bellman equations have a unique solution. We can find it by treating the right-hand side as an update rule, given any estimate of V^*, plug it in to get a better one. Each application is a *Bellman update*, and repeating this process, called *value iteration*, is guaranteed to converge. The reason is γ, as any error in the current estimate gets multiplied by $\gamma < 1$ each round, so the gap between the estimate and the true V^* shrinks by at least a factor of γ with every update. The closer γ is to 1, the slower the shrinkage, but it always shrinks. This requires knowing T and R upfront, though. When the agent has to learn from experience instead, we need a different approach, which the next lecture will cover.

As an example, let $\gamma = 0.9$ and initialize $V^0(s) = 0$ for every state. The first Bellman update substitutes V^0 into the right-hand side of the Bellman equation:

$$V^1(\text{Writing}) = \max\begin{cases} 1 + 0.9 \cdot 0 = 1 & [\text{Focus}] \\ 2 + 0.9(0.5 \cdot 0 + 0.5 \cdot 0) = 2 & [\text{Browse}] \end{cases} = 2$$

$$V^1(\text{Googling}) = \max\begin{cases} 1 + 0.9(0.5 \cdot 0 + 0.5 \cdot 0) = 1 & [\text{Focus}] \\ -10 + 0.9 \cdot 0 = -10 & [\text{Browse}] \end{cases} = 1$$

$$V^1(\text{DS}) = -10 + 0.9 \cdot 0 = -10$$

With V^1 in hand, a second update gives:

$$V^2(\text{Writing}) = \max\begin{cases} 1 + 0.9 \cdot 2 = 2.8 & [\text{Focus}] \\ 2 + 0.9(0.5 \cdot 2 + 0.5 \cdot 1) = 3.35 & [\text{Browse}] \end{cases} = 3.35$$

$$V^2(\text{Googling}) = \max\begin{cases} 1 + 0.9(0.5 \cdot 2 + 0.5 \cdot 1) = 2.35 & [\text{Focus}] \\ -10 + 0.9 \cdot (-10) = -19 & [\text{Browse}] \end{cases} = 2.35$$

$$V^2(\text{DS}) = -10 + 0.9 \cdot (-10) = -19$$

Each round the estimates climb toward the true V^*. To see where those values land, we can solve the Bellman equations directly as a linear system. DS is absorbing, so its equation has one unknown:

$$\begin{aligned} V^*(\text{DS}) &= -10 + 0.9 \cdot V^*(\text{DS}) \\ 0.1 \cdot V^*(\text{DS}) &= -10 \\ V^*(\text{DS}) &= -100 \end{aligned}$$

For Writing and Googling we can do something similar, but first we have to deal with the max. The full Bellman equation for Writing looks like:

$$V^*(\text{Writing}) = \max \begin{cases} 1 + 0.9 \cdot V^*(\text{Writing}) & [\text{Focus}] \\ 2 + 0.9(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) & [\text{Browse}] \end{cases}$$

The max makes this non-linear since we cannot collect terms and solve while there is still a choice embedded in the equation. But if we already know the optimal action at each state, we can drop the max and just write down the equation for that action. We know Browse is optimal from Writing and Focus is optimal from Googling, so the equations become:

$$\begin{aligned} V^*(\text{Writing}) &= 2 + 0.9(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) \\ V^*(\text{Googling}) &= 1 + 0.9(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) \end{aligned}$$

Both equations share the same weighted average, so let $L = 0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})$:

$$\begin{aligned} L &= 0.5(2 + 0.9L) + 0.5(1 + 0.9L) = 1.5 + 0.9L \\ 0.1L &= 1.5 \\ L &= 15 \end{aligned}$$

Substituting back: $V^*(\text{Writing}) = 2 + 0.9(15) = 15.5$ and $V^*(\text{Googling}) = 1 + 0.9(15) = 14.5$.

There is something circular about this. We needed to know the optimal policy (Browse from Writing, Focus from Googling) before we could drop the max and write a linear system, but the whole point was to find the optimal policy in the first place. In this example we could figure out the policy by inspection, but in general that is not possible. Value iteration sidesteps this as it never assumes a fixed policy. It keeps the max in place and just repeatedly applies the full Bellman equation as an update rule. Each round the estimates get closer to V^*, and the max at each step naturally selects whichever action looks best given the current estimates.

The absolute values take many rounds to converge, but the relative ordering of actions is often correct early on. After just one update, the Q-values from Writing are 2 for Browse and 1 for Focus, so Browse wins. The true Q-values are:

$$\begin{aligned} Q^*(\text{Writing}, \text{Browse}) &= 15.5 \\ Q^*(\text{Writing}, \text{Focus}) &= 1 + 0.9 \cdot 15.5 = 14.95 \end{aligned} \tag{10}$$

So Browse still wins. From Googling, round one gives 1 for Focus and -10 for Browse. The true values are:

$$Q^*(\text{Googling}, \text{Focus}) = 14.5$$
$$Q^*(\text{Googling}, \text{Browse}) = -10 + 0.9(-100) = -100 \tag{11}$$

So Focus wins by an even wider margin. Value iteration was converging on the right policy immediately, however it just needs more rounds to get the magnitudes right.

Expectimax

The structure of the Bellman equation should look familiar. The $\max_a$ is an agent node; the agent picks the action that maximizes its value. The $\sum_{s'} T(s, a, s')[...]$ is a chance node, where the environment picks the next state according to the transition probabilities. This is the expectimax tree from earlier in the course, but written as a set of equations over all states simultaneously rather than a tree expanded from a single root. Importantly, the Bellman equation handles cycles elegantly. While a tree would need to unroll cycles into infinitely deep recursive loops, the Bellman equations form a finite system, with one equation per state, that can be solved all at once regardless of cycles.

To illustrate this, look at the tree in Figure 3 and compare it to our transition model in figure 2. In this tree, the circle nodes are *Q-states*, with each one representing a specific (s, a) pair, the moment after the agent has committed to an action but before the environment has revealed where it lands. This is the same structure as the chance nodes from expectimax, as the agent controls the edge going into a Q-state (by choosing an action), and the environment controls the edges going out of it (by sampling s' according to $T(s, a, X)$). Agent nodes take a max over their children just as in expectimax, and Q-states take the expectation over theirs.

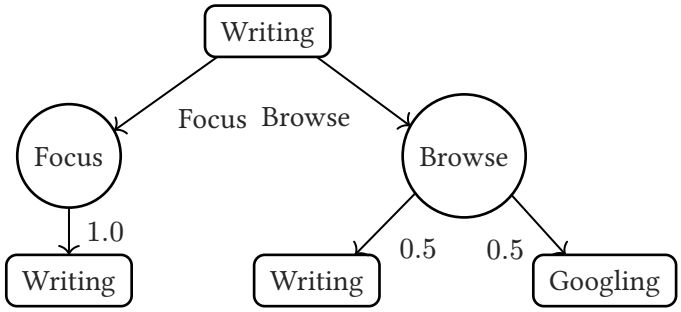

Figure 5: Expectimax trees for Writing. Square nodes are agent (max) nodes, circle nodes are Q-states where the environment resolves the outcome.

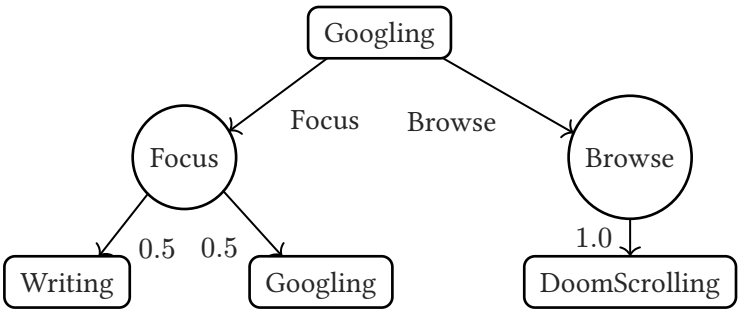

Figure 6: Expectimax tree for Googling.

Applying Bellman

Our example leaves us with three possible outcomes, Writting, Googling and DoomScrolling. With in mind, lets expand the Bellman equation directly for each non-terminal state:

$$V^*(\text{Writing}) = \max\begin{cases} 1 + \gamma V^*(\text{Writing}) & [\text{Focus}] \\ 2 + \gamma(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) & [\text{Browse}] \end{cases}$$

$$V^*(\text{Googling}) = \max\begin{cases} 1 + \gamma(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) & [\text{Focus}] \\ -10 + \gamma V^*(\text{DS}) & [\text{Browse}] \end{cases}$$

We already know Browse dominates from Writing and Focus dominates from Googling for all $\gamma \in [0, 1)$, so the max resolves and we get the same equations from the previous section[6]. The Bellman equation did not tell us anything we did not already know, but it gives us a general procedure that works even when the optimal policy is not obvious by inspection.

[6] Equation 5 from the analysis of the discussion. Page 6.

So now if we combined the trees from figures three and four, starting from Writing and expanding two full rounds we end up with the tree below:

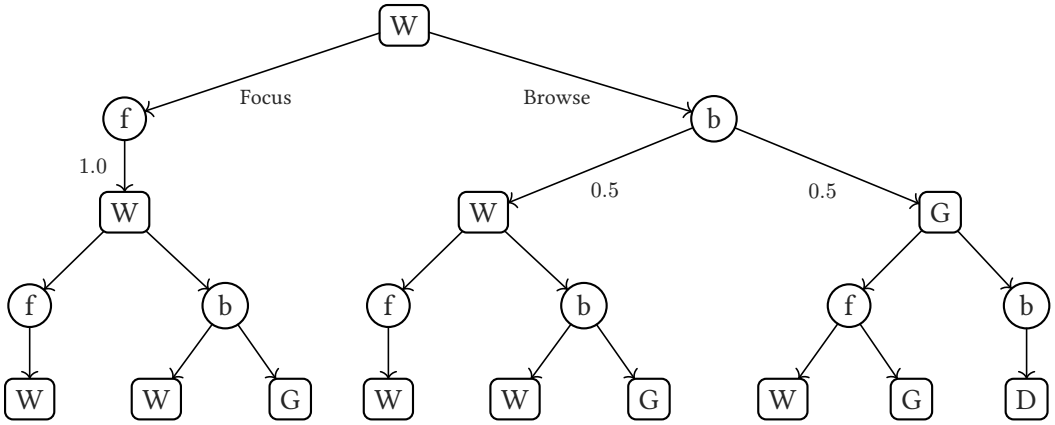

Figure 7: Depth-2 expectimax tree rooted at Writing. Square = agent node, circle = Q-state. W = Writing, G = Googling, D = DS. Probabilities on edges match Table 1; and they are omitted at depth 2 for clarity.

Policy Iteration

Value iteration keeps the max in place and converges by repeating Bellman updates. Policy iteration takes a different route: it alternates between two steps.

Policy evaluation takes a fixed policy π and solves for V^π exactly. With the policy fixed, there is no max, every state has a single prescribed action, so the Bellman equations become a linear system solvable in one shot (exactly as we did earlier when deriving the true values).

Policy improvement then looks at those values and asks: given V^π, is there any state where a different action would do better? Formally, for each state compute:

$$\pi'(s) = \arg\max_a \sum_{s'} T(s,a,s') \Big[R(s,a,s') + \gamma V^{\pi(s')} \Big] \tag{12}$$

If $\pi' = \pi$, the policy is already optimal and we stop. Otherwise set $\pi = \pi'$ and repeat.

To see this in action, start with $\pi^0 = \{\text{Writing} : \text{Focus}, \text{Googling} : \text{Focus}\}$, the most conservative possible policy. Evaluating π^0 (with $\gamma = 0.9$) gives a linear system with no max:

$$V^{\pi^0}(\text{Writing}) = 1 + 0.9 \cdot V^{\pi^0}(\text{Writing}) \Rightarrow V^{\pi^0}(\text{Writing}) = 10$$
$$V^{\pi^0}(\text{Googling}) = 1 + 0.9\Big(0.5 \cdot 10 + 0.5 \cdot V^{\pi^0}(\text{Googling})\Big) \Rightarrow V^{\pi^0}(\text{Googling}) = 10$$

Now improve. For each state, compute Q-values using V^{π^0}:

$$Q(\text{Writing}, \text{Focus}) = 1 + 0.9 \cdot 10 = 10$$
$$Q(\text{Writing}, \text{Browse}) = 2 + 0.9(0.5 \cdot 10 + 0.5 \cdot 10) = 11 \rightarrow \text{Browse wins}$$
$$Q(\text{Googling}, \text{Focus}) = 1 + 0.9(0.5 \cdot 10 + 0.5 \cdot 10) = 10$$
$$Q(\text{Googling}, \text{Browse}) = -10 + 0.9 \cdot (-100) = -100 \rightarrow \text{Focus wins}$$

The improved policy is $\pi^1 = \{\text{Writing} : \text{Browse}, \text{Googling} : \text{Focus}\}$. Evaluating π^1 gives $V^{\pi^1}(\text{Writing}) = 15.5$ and $V^{\pi^1}(\text{Googling}) = 14.5$ (the same linear system we solved before). Running improvement again produces the same policy, so we stop. Policy iteration found the optimal policy in a single iteration.

Policy iteration and value iteration always converge to the same V^*, but they get there differently. Value iteration refines value estimates gradually, with the policy implicitly defined by the max at each step. Policy iteration commits to an explicit policy, evaluates it exactly, then jumps to a strictly better one. Each improvement step is guaranteed to not get worse, and since there are only finitely many policies, the process must terminate.